PRODUCT REQUIREMENTS DOCUMENT

Submission Tracker

A shared workspace for account managers to log, see, and manage candidate submissions

- **Owner: Priyam Pathak**
- **Status:** [Draft] ready for leadership review
- **Last updated:** 9 Sep 2026
- **Reviewers needed:** Engineering, Sales, Ops, Finance

1. Summary

Account managers (AMs) currently track candidate submissions in a shared spreadsheet. It's breaking down: AMs double-submit candidates because they can't see what teammates already did, clients ask AMs for status and AMs have to go chase someone else to find out, and leadership can't get conversion numbers out of a spreadsheet.

We're building a Submission Tracker module: one shared place to log a submission, see its live status, and get warned before creating an accidental duplicate. Reporting on conversion rates is the long-term goal, but it depends on data we don't have confirmed access to yet, so it is not in v1(version 1).

Why this matters now - The AM team is repeating manual work, clients are getting slower answers than they should, and leadership is flying blind on performance. All three get worse as submission volume grows, this is a scaling problem, not a nice-to-have.

2. The Problem

Three distinct problems, all caused by the same root issue: submissions live in a spreadsheet with no shared, structured, real-time view.

- **Duplicate submissions:** Two AMs can submit the same candidate to the same job without knowing the other one already did, because there's no shared visibility.
- **Slow status answers:** When a client asks “where's my candidate?”, the AM doesn't know either, they have to go ask engineering or ops and get back to the client later.
- **No reporting:** Leadership wants submission-to-placement conversion by AM and by client. A spreadsheet built for logging, not reporting, can't produce that reliably.

Example: Two AMs, Priya and Raj, both work with the same client. Priya submits candidate “Anita Rao” to Req #204 on Monday. Raj, unaware, submits the same candidate to the same req on Wednesday. The client now has two submissions for one person and starts asking questions, a bad look for BYLD.

3. Goals & Success Metrics

Each goal below is written so it can be measured, “improve visibility” isn't measurable, the numbers below are.

- **Goal 1:** Cut accidental duplicate submissions
  - **Metric:** ***N*** of submissions per month flagged with duplicate-reason “accidental”
  - **How we measure it:** Compare against a 4-week baseline pulled from the old spreadsheet
  - **v1 target:** ≥ 70% reduction within 30-50 days of launch

- **Goal 2:** Cut time for AMs to answer client status questions
  - **Metric:** Average time an AM takes to answer a status question

- **How we measure it:** Lightweight AM survey before and after launch, we can't instrument client conversations directly
- **v1 target:** ≥ 50% faster, self-reported

- **Goal 3:** Get AMs fully off the spreadsheet
  - **Metric:** % of new submissions logged in the tool vs. the old spreadsheet
  - **How we measure it:** In-tool submission count ÷ (in-tool + spreadsheet count) per week
  - **v1 target:** ≥ 90% within 4 weeks of launch

- **Goal 4:** Enable conversion reporting (v2 goal, tracked from day one)
  - **Metric:** Submission-to-placement conversion % by AM and by client
  - **Status:** Not scored in v1, blocked on the finance-system data dependency. Data model is built v1-ready so v2 is fast to add, not a rebuild.

4. User Personas - Who Uses This?

Three roles interact with this tool. Each needs something different from it.

Account Manager - *primary, daily user*

- **Goals:** Submit candidates fast, keep clients updated, avoid embarrassing duplicate submissions
- **Pain today:** No shared visibility into teammates' submissions; has to chase ops/engineering for status, re-checks a messy spreadsheet before every submission
- **Needs from this tool:** Log a submission in seconds, see every submission across their candidates and clients, get warned instantly if a candidate is already submitted

Ops / Delivery - *updates submission status as things move*

- **Goals:** Keep submission status accurate and current without digging through email threads
- **Pain today:** Status updates today live in scattered emails/Slack messages, not one system of record
- **Needs from this tool:** A simple, fast way to change a submission's status and see history of what changed and when

Sales / Leadership - *oversight, not daily hands-on use*

- **Goals:** Trust that duplicate submissions to clients are rare and visible when they happen; eventually see conversion numbers
- **Pain today:** No audit trail today, duplicate incidents are only discovered when a client complains
- **Needs from this tool:** A record of every flagged duplicate and why it happened, conversion reporting once the finance-data question is resolved

Note -
Clients won't access the tool in v1. It's strictly for Account Managers to improve response times with centralised data. This is intentional, not an oversight. A client portal is planned for v2 in Section 7 below. If early access is desired, it affects the scope significantly.

5. Constraints That Shape This Plan

Three real constraints changed what v1 actually is. Ignoring them would produce a plan nobody can deliver.

5.1 Engineering capacity is small, not "the full squad"

Leadership has been talking about "next sprint" assuming the whole squad is free. In reality, one engineer, part-time, is available for the next two sprints because the rest of the team is on a billing migration.

What this means for scope - v1 has to be small enough for roughly half of one engineer's time across two sprints. That rules out real-time integrations, dashboards, and anything needing a second engineer to parallelize. See Section 7 for exactly what fits.

5.2 The finance-system data dependency is unconfirmed

Leadership's conversion-rate reporting depends on a "placement-confirmed" event that today only exists inside the finance system. Nobody has confirmed if that data can be exported, how often, in what format, or who owns granting access.

What this means for scope - We cannot commit to a conversion-rate dashboard in v1. We can commit to building the Submission Tracker's data model so a status of "Placed" is already captured manually, so the moment the finance data is confirmed, v2 reporting is fast to add, not a rebuild.

5.3 Sales and AMs disagree on duplicate handling

Sales wants duplicates blocked outright. Two AMs separately said a hard block would break real, legitimate cases, resubmitting a candidate for a different role, or resubmitting after a rejection where the situation changed. Addressed head-on in Section 6.

6. Key Decision: How We Handle Duplicate Submissions?

Decision - We will not hard-block duplicate submissions. We will warn the AM at the point of submission, show them exactly what was already submitted (by whom, when, current status), and require them to pick a reason before proceeding. A true silent duplicate, same candidate, same requisition, no reason given, prior submission still active, now is the only thing that gets blocked.

Why not a hard block, since Sales asked for one

The actual problem reported wasn't "AMs are deliberately abusing the system", it was "AMs can't see what teammates already did." A hard block solves a policy problem we don't have and creates a workflow problem we do have: two AMs already flagged real cases (different role, changed circumstances after rejection) where resubmitting is the right call, not a mistake.

A hard block also can't tell the difference between an honest mistake and a deliberate, valid resubmission, it has no way to know the AM's intent. A warning-plus-reason model fixes the actual root cause (no visibility) while leaving room for judgment calls a machine shouldn't be making alone.

Why this is still safe for Sales

Every override is logged with a reason and a name attached. Sales gets an audit trail of every resubmission and why it happened , more useful for spotting a real problem pattern than a blunt block would be, and it's a simple filter on the submission list, not a separate tool.

Example: Raj tries to submit Anita Rao to Req #204. The tool shows: "Already submitted by Priya on Mon, 3 days ago - status: Client Review." Raj must pick a reason ("Resubmitting for a different role," "Circumstances changed after rejection," "Other - explain") or cancel. If he picks a reason, the submission goes through and is tagged as a flagged duplicate for Sales' audit view.

7. Scope: v1 and v2

Everything below, what we're building now, and what we're deliberately not, traces back to the constraints in Section 5. v1 is scoped tight on purpose: to what roughly half of one engineer can realistically ship in two sprints, and to what solves the two most painful, most-reported problems (no visibility, slow status answers).

v1 - Building now

Everything below, what we're building now, and what we're deliberately not, traces back to the constraints in Section 5. v1 is scoped tight on purpose: to what roughly half of one engineer can realistically ship in two sprints, and to what solves the two most painful, most-reported problems (no visibility, slow status answers).

- **Feature:** Log a submission (candidate + requisition + client + AM + date)
  - **Why:** This is the shared record that fixes “no visibility”, the root cause behind everything else in Section 2.
- **Feature:** Duplicate-submission warning at the point of logging
  - **Why:** Directly targets the #1 reported problem, at low engineering cost - a lookup and a modal, not a new system.

- **Feature:** Manually-set status field (Submitted → Client Review → Interview → Offer/Final Call → Placed / Rejected / Withdrawn)
  - **Why:** Lets an AM answer a client instantly instead of chasing someone. No integration needed, a person just updates a dropdown.
- **Feature:** List view with filters (by AM, client, requisition, status)
  - **Why:** This is literally the screen an AM opens ten times a day. Needed for the tool to be usable, not a stretch feature.

- **Feature:** Basic manual counts (submissions by status, by AM) on the same list view
  - **Why:** Gives leadership something better than the spreadsheet immediately, without needing the finance data or a dashboard build.

v2 / Deferred - not building yet

- **Feature:** Live conversion-rate dashboard
  - **Why deferred:** The underlying event doesn't have a confirmed data source yet.
  - **Blocked by:** Finance-system data access (Section 5.2)
- **Feature:** Automated placement-data pull from the finance system
  - **Why deferred:** Same reason, no confirmed pipeline, format, or access owner.
  - **Blocked by:** Finance-system data access
- **Feature:** Automatic blocking of duplicates
  - **Why deferred:** Would break real AM workflows - see Section 6.
  - **Blocked by:** A product decision, not a technical limitation
- **Feature:** Client-facing status portal
  - **Why deferred:** Not requested in the brief; bigger scope than one part-time engineer can take on.
  - **Blocked by:** Engineering capacity (Section 5.1)
- **Feature:** Notifications / alerts on status change
  - **Why deferred:** Nice-to-have; doesn't directly solve one of the three reported problems in Section 2.
  - **Blocked by:** Engineering capacity

8. User Stories & Flows

Written so an engineer can size each one on its own, each has a trigger, a behavior, and a clear “done” condition.
A note on sizing: each story uses relative t-shirt sizing (S/M/L), not numeric story points (1, 2, 3, 5, 8…). Numeric points are calibrated against a team's past sprint velocity, this is a net-new module with no velocity history yet, so a relative size is the more honest estimate right now. Sizes are relative for a single engineer on a net-new module with nothing to build on (S ≈ 1–2 days, M ≈ 3–5 days) — confirm with engineering before sprint planning.

8.1 Core User Flows

Each user flow represents the precise sequence a user follows, transitioning from one screen to another. These flows are directly mapped to the stories in section 8.2 and are identified under each flow below.

Flow 1 - Create Submission (maps to Story 1)

1. AM opens the Submission Tracker and clicks “New Submission.”
2. AM selects a requisition; the client name auto-fills based on this selection.
3. AM enters candidate details (name, email, or phone) and confirms the submission date, which defaults to today.

4. AM clicks “Save.” The system checks for a possible duplicate before saving (this check is detailed in Flow 2).

5. If no duplicate is found, the submission saves and appears immediately at the top of the shared list.

Outcome: A new, visible submission record is created that any AM or ops user can see, eliminating the need for spreadsheets.

Flow 2 - Duplicate Submission (maps to Story 2)

1. AM fills out and submits the form following the steps in Flow 1.
2. The system checks the candidate's email/phone against the requisition for an existing submission before saving.
3. If a match is found, a warning appears, displaying who submitted it, when, and its current status.
4. AM selects a reason (“Different role,” “Circumstances changed after rejection,” “Other - explain”) to proceed, or clicks Cancel to stop.
5. If a reason is given, the submission saves and is tagged “Flagged duplicate” for Sales' audit view (Story 6). If cancelled, nothing is saved, and the AM returns to the form.

Outcome: Submissions are never silently duplicated. Each duplicate is either halted or logged with a reason and a name.

Flow 3 - Update Submission Status (maps to Story 3)

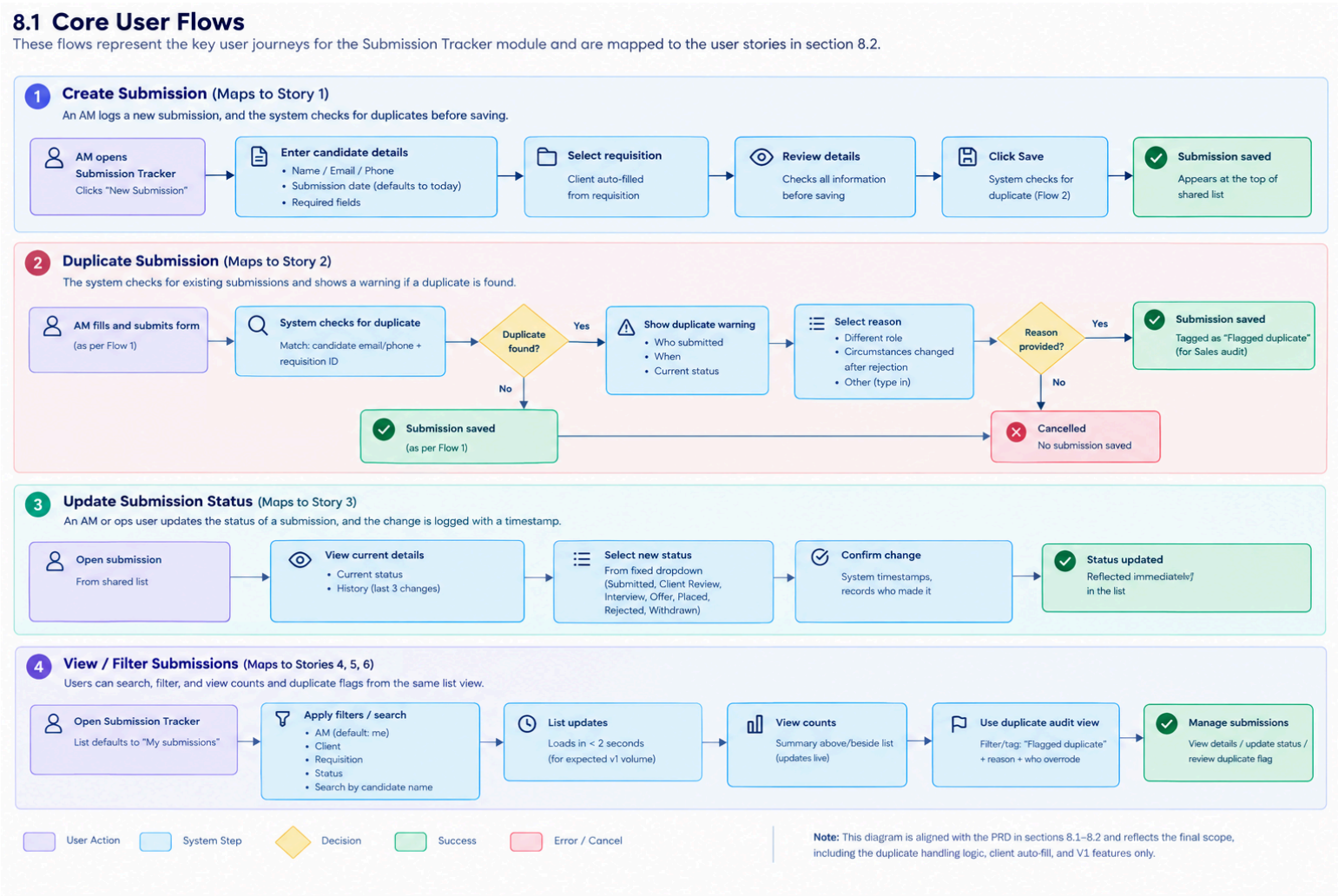
1. AM or ops user opens a submission from the shared list.
2. They open the status dropdown (Submitted, Client Review, Interview, Offer, Placed, Rejected, Withdrawn).
3. They select the new status and confirm the change.
4. The system timestamps the change, records who made it, and adds it to the submission's status history.
5. The updated status is reflected immediately in the shared list—no manual refresh needed.

Outcome: An AM can quickly respond to client inquiries about a candidate's status by checking the tool, without involving ops or engineering.

Flow 4 - View / Filter Submissions (maps to Stories 4, 5, 6)

1. AM opens the Submission Tracker; the list defaults to “my submissions.”
2. AM applies a filter (client, requisition, or status) or searches by candidate name.
3. The list updates to match the criteria within 2 seconds for expected v1 volume.
4. AM can view basic counts (e.g., submissions by status) above the list without leaving the page.
5. Sales/ops can apply the “Flagged duplicate” filter to see every override and its reason, within the same view—no separate tool required.

Outcome: One screen provides a comprehensive view of submissions across roles, allowing AMs, ops, and Sales to efficiently track and manage submissions.



8.2 User Stories & Acceptance Criteria

Each user story is associated with one of the flows above. Acceptance criteria define what “done” means, ensuring clarity for engineers to build without needing further clarification.

Story 1 - Log a submission (Size: M - Flow 1)

As an AM, I log a new submission so my team can see it.

- Form includes candidate name + contact, requisition, client (auto-filled from requisition), and submission date (defaults to today).
- On save, it appears immediately in the shared list.
- Required fields must be completed before saving.

Story 2 - Duplicate warning (Size: M - Flow 2)

As an AM, I receive a warning if I'm about to submit a candidate already submitted to the same requisition.

- Match checked by candidate email/phone + requisition ID before save.
- Warning displays who submitted, when, and the current status.
- AM must select a reason from a fixed list, or type one under “Other,” to proceed.
- If the prior submission is Rejected/Withdrawn and older than 90 days, the info is shown but a reason is not required — low-risk case.

Story 3 - Update status (Size: S - Flow 3)

As an AM or ops user, I update a submission's status as it progresses.

- Status is a fixed dropdown (Submitted, Client Review, Interview, Offer, Placed, Rejected, Withdrawn).
- Every change is timestamped and shows who made it.
- Status history is visible on the submission (at least the last 3 changes).

Story 4 — Filter and search (Size: S - Flow 4)

As an AM, I filter and search the submission list to quickly find needed information.

- Filters: by AM (default: me), by client, by requisition, by status.
- Search by candidate name.
- List loads in under 2 seconds for expected v1 volume.

Story 5 — Basic counts (Size: S - Flow 4)

As a leader, I see basic counts without opening a spreadsheet.

- Simple count summary above or beside the list view, not a separate dashboard page.
- Numbers update live as statuses change, no separate refresh or export step.

Story 6 — Duplicate audit view (Size: S - Flow 4)

As Sales, I can see which submissions were flagged as duplicates and why.

- Filter/tag on the existing list view: “Flagged duplicate” + reason + who overrode it.
- No separate tool needed, same list view, just filterable.

Sizes are relative for a single engineer working on a new module with no existing framework (S ≈ 1–2 days, M ≈ 3–5 days). Engineering confirmation is advised before sprint planning.

8.3 Definition of Done

The Definition of Done applies to every story and ensures that each one is fully completed before being considered shippable.

- All acceptance criteria for that story are met.
- Code is reviewed and merged (even a part-time solo engineer should self-review or get a quick async review from another team member).
- Manually tested on staging before going live, no dedicated QA assumed for v1 given capacity constraints in Section 5.1.

- No known P0/P1 bugs open against it (P0 = breaks core flow, P1 = incorrect data shown).

Story 1 - Log a submission (Size: M)

As an AM, I log a new submission so my team can see it.

- Form: candidate name + contact, requisition, client (auto-filled from requisition), submission date (defaults to today)

- On save, it appears immediately in the shared list
- Required fields are enforced - can't save a blank submission

Story 2 - Duplicate warning (Size: M)

As an AM, I get warned if I'm about to submit a candidate already submitted to the same requisition.

- Match is checked by candidate email/phone + requisition ID before save
- Warning shows: who submitted, when, and the current status
- AM must pick a reason from a fixed list, or type one under “Other,” to proceed

- If the prior submission is Rejected/Withdrawn and older than 90 days, show the info but don't force a reason, low-risk case

Story 3 - Update status (Size: S)

As an AM or ops user, I update a submission's status as it moves forward.

- Status is a fixed dropdown (Submitted, Client Review, Interview, Offer, Placed, Rejected, Withdrawn)
- Every change is timestamped and shows who made it
- Status history is visible on the submission (at least the last 3 changes)

Story 4 - Filter and search (Size: S)

As an AM, I filter and search the submission list so I can find what I need fast.

- Filters: by AM (default: me), by client, by requisition, by status
- Search by candidate name
- List loads in under 2 seconds for expected v1 volume

Story 5 - Basic counts (Size: S)

As a leader, I see basic counts without opening a spreadsheet.

- Simple count summary above or beside the list view, not a separate dashboard page
- Numbers update live as statuses change - no separate refresh or export step

Story 6 - Duplicate audit view (Size: S)

As Sales, I can see which submissions were flagged as duplicates and why.

- Filter/tag on the existing list view: “Flagged duplicate” + reason + who overrode it
- No separate tool needed - same list view, just filterable

Sizes are relative for a single engineer on a net-new module with nothing to build on (S ≈ 1–2 days, M ≈ 3–5 days) - confirm with engineering before sprint planning.

9. Edge Cases & Open Questions

These need a named stakeholder's answer before engineering starts. Where useful, a recommended default is included so the team isn't blocked waiting, but these still need to be confirmed, not assumed.

- **Same person, different contact info:** Two candidate submissions use different contact details for the same real person (e.g. a personal email one time, a work email another), the match logic misses it and lets a real duplicate through.
  - Why it matters: this is the most likely way the duplicate warning quietly fails in practice.
  - Recommended default: match on email OR phone (not both required), and let AMs manually flag/merge a missed duplicate if they spot one.
  - Who confirms: Product + Engineering
- **Old rejection, role reopens later:** A client reopens a role and an AM resubmits a candidate who was rejected 8 months ago - does the system still treat that as a duplicate worth warning about?
  - Why it matters: an old, stale warning trains AMs to click through warnings without reading them, which defeats the feature.
  - Recommended default: no warning past 90 days on a closed (Rejected/Withdrawn) submission, already reflected in Story 2.
  - Who confirms: Sales + AM team
- **Spreadsheet migration:** Do we migrate the historical spreadsheet data into the tool, or start clean at launch?
  - Why it matters: migrating means duplicate-warnings work from day one; starting clean means a faster launch but weaker warnings for the first few weeks.
  - Recommended default: migrate the last 90 days only (matches the duplicate-warning window in Story 2) - full history isn't needed for the warning to work.
  - Who confirms: Product + Leadership

10. If Scope Has to Shrink Further

Using MoSCoW - simple, and easy for non-PM stakeholders (Sales, Finance, Ops) to follow without translation.

- **Must-have:** Log a submission + shared list view
  - Without this, nothing else exists. This alone already fixes “no visibility” - the root cause behind two of the three problems in Section 2.
- **Must-have:** Duplicate-submission warning
  - The single most-repeated complaint in the brief. Cheap to build relative to its impact - a lookup, not a new system.

- **Should-have:** Manual status updates + status history
  - High value for the “clients keep asking for status” problem, but the tool still has value without it - AMs can still avoid duplicates and see what's logged.
- **Should-have:** Filters/search on the list
  - Makes the tool usable at scale, but a flat, unfiltered list still technically works for a small team short-term.
- **Could-have:** Basic status counts for leadership
  - A nice early win for leadership buy-in, but not core to solving an AM's day-to-day problem.
- **Won't-have (v1):** Everything in Section 7's deferred list
  - Either blocked by a dependency we don't control (finance data) or bigger than current engineering capacity supports.

If sprint capacity turns out even smaller than planned, the two Must-haves are the floor, they're the only combination that still solves a real, reported problem on their own.

Using RICE cross-check on the two Must-haves

Feature 1: Log a Submission + Shared List View

Reach: Highest reach, impacting every Account Manager (AM) and submission weekly.

Impact: High, as it's foundational to the system.

Confidence: High, due to low technical risk and addressing the root cause.

Effort: Medium (Story 1, size M).

Net Read: Highest RICE score, marked as "Must-have."

Feature 2: Duplicate-Submission Warning

Reach: Affects every AM and submission attempt.

Impact: High, addressing a frequent complaint.

Confidence: Medium-high, reliant on data quality.

Effort: Medium (Story 2, size M).

Net Read: Second-highest RICE score, prioritized after the first feature.

Conclusion

RICE and MoSCoW agree on these priorities, ensuring maximum impact and efficiency.

11. Risks & Dependencies

- **Risk:** Finance-system data access takes longer than expected, or isn't possible in the current form
  - Impact: conversion reporting stays blocked indefinitely, leadership's original ask goes unmet
  - Mitigation: data model is built v1-ready for a “Placed” status now, so v2 only needs the pipeline, not new schema work. Flag this timeline risk to leadership explicitly - don't let “next sprint” expectations persist.
- **Risk:** The one part-time engineer gets pulled further onto the billing migration
  - Impact: Sprint 1 scope (Section 7) slips
  - Mitigation: keep v1 scope small enough that even a partial sprint ships the two Must-haves (Section 10).
- **Risk:** AMs don't adopt the tool and keep using the spreadsheet out of habit
  - Impact: the core problem isn't actually solved even if the tool ships
  - Mitigation: make logging faster than the spreadsheet (pre-filled fields from requisition data) and track the adoption metric from Section 3 starting week one.

Appendix: Minimal Data Model (for engineering reference)

Not a full schema, just the core fields v1 needs, so engineering can size Section 8's stories accurately.

- **Candidate:** candidate_id, name, email, phone
- **Requisition:** requisition_id, title, client_id
- **Submission:** submission_id, candidate_id, requisition_id, submitted_by (AM), submitted_at, status, status_history[], duplicate_flag (bool), duplicate_reason (nullable)
- **Client:** client_id, name

The status_history and duplicate_flag/reason fields exist specifically so Section 6's decision and Section 3's metrics are measurable from day one, not bolted on later.